



DATA WAREHOUSE DESIGN AND IMPLEMENTATION FOR OLIST E-COMMERCE



MAY 1, 2026

FADI AL-ARIQI

1. Introduction

This document details the design and implementation of a Star Schema Data Warehouse for the Olist e-commerce dataset. The primary objective is to create a robust analytical platform that supports comprehensive reporting and business intelligence across various operational areas, including sales, payments, delivery, customer reviews, and marketing leads.

2. Architecture Diagram

The overall architecture of the solution involves extracting data from the SQLite OLTP database, transforming it into a Star Schema, and loading it into a PostgreSQL Data Warehouse. The reporting layer will then query this Data Warehouse for analytical insights.

[Image failed to load: Olist Data Warehouse Architecture]

3. Star Schema Design Principles

The data warehouse is meticulously designed following the principles of a Star Schema, which is optimized for query performance and ease of understanding for analytical workloads. Key design tenets include:

- **Fact tables are isolated:** Fact tables are not directly linked to each other. All relationships are established exclusively between fact tables and their corresponding dimension tables.
- **Single business process per fact table:** Each fact table is dedicated to a distinct business process, ensuring a clear and consistent grain for each analytical domain.
- **Strict adherence to grain:** The granularity of each fact table is precisely defined and maintained (e.g., fact_sales is at the order item level).
- **Degenerate Dimensions:** Transactional attributes that do not belong to any specific dimension, but are crucial for analysis, are included directly in the fact tables as degenerate dimensions (e.g., order_id, mql_id).

4. Data Model Explanation

4.1. Fact Tables

4.1.1. fact_sales

This fact table captures sales information at the individual order item level, enabling granular analysis of product sales, pricing, and associated costs.

Column Name	Description	Type
<u>sales_key</u>	Primary Key (PK): Unique identifier for each sales record.	PK
<u>order_id</u>	Degenerate Dimension: Original order identifier from the OLTP system.	Degenerate
<u>order_item_id</u>	Degenerate Dimension: Identifier for each specific item within an order.	Degenerate
<u>product_key</u>	Foreign Key (FK): Links to the <u>dim_product</u> table.	FK
<u>customer_key</u>	Foreign Key (FK): Links to the <u>dim_customer</u> table.	FK
<u>seller_key</u>	Foreign Key (FK): Links to the <u>dim_seller</u> table.	FK
<u>order_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, representing the date the order was placed.	FK
<u>shipping_limit_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, indicating the shipping limit date for the item.	FK
<u>quantity</u>	The quantity of the product in the order item.	Numeric
<u>unit_price</u>	The unit price of the product at the time of sale.	Numeric
<u>freight_value</u>	The freight value (shipping cost) associated with the order item.	Numeric
<u>total_price</u>	The total price for the order item, calculated as <u>quantity</u> * <u>unit_price</u> + <u>freight_value</u> .	Numeric

4.1.2. fact payments

This fact table records detailed payment transactions, allowing for analysis of payment methods, installment plans, and payment values for each order.

Column Name	Description	Type
<u>payment_key</u>	Primary Key (PK): Unique identifier for each payment record.	PK
<u>order_id</u>	Degenerate Dimension: Original order identifier.	Degenerate

Column Name	Description	Type
<u>customer_key</u>	Foreign Key (FK): Links to the <u>dim_customer</u> table.	FK
<u>payment_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, representing the date the payment was made.	FK
<u>payment_type_key</u>	Foreign Key (FK): Links to the <u>dim_payment_type</u> table.	FK
<u>payment_value</u>	The monetary value of the payment.	Numeric
<u>payment_installments</u>	The number of installments chosen for the payment.	Numeric

4.1.3. fact_delivery

An accumulating snapshot fact table that tracks the entire lifecycle of an order, from its placement to customer delivery. It includes calculated metrics for delivery time and delays.

Column Name	Description	Type
<u>delivery_key</u>	Primary Key (PK): Unique identifier for each delivery record.	PK
<u>order_id</u>	Degenerate Dimension: Original order identifier.	Degenerate
<u>customer_key</u>	Foreign Key (FK): Links to the <u>dim_customer</u> table.	FK
<u>seller_key</u>	Foreign Key (FK): Links to the <u>dim_seller</u> table.	FK
<u>order_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, representing the date the order was placed.	FK
<u>approved_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, representing the date the order was approved.	FK
<u>delivered_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, representing the date the order was delivered to the customer.	FK
<u>estimated_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, representing the estimated delivery date.	FK
<u>delivery_time_days</u>	Calculated: The number of days between order approval and customer delivery.	Numeric

Column Name	Description	Type
<u>delivery_delay_days</u>	Calculated: The number of days the delivery was delayed, if <u>delivered_date</u> is after <u>estimated_date</u> .	Numeric
<u>is_delayed</u>	A flag (1 for delayed, 0 for on-time) indicating if the delivery exceeded the estimated date.	Boolean

4.1.4. fact_reviews

This fact table stores customer review information, allowing for analysis of customer satisfaction and feedback at the order level.

Column Name	Description	Type
<u>review_key</u>	Primary Key (PK): Unique identifier for each review record.	PK
<u>order_id</u>	Degenerate Dimension: Original order identifier.	Degenerate
<u>customer_key</u>	Foreign Key (FK): Links to the <u>dim_customer</u> table.	FK
<u>review_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, representing the date the review was created.	FK
<u>review_score</u>	The numerical score given by the customer (typically 1-5).	Numeric
<u>has_comment</u>	A flag (1 if a comment was provided, 0 otherwise) indicating the presence of a review message.	Boolean

4.1.5. fact_leads

This fact table tracks marketing qualified leads (MQLs) and their progression through the sales funnel, supporting analysis of lead sources, business segments, and conversion rates.

Column Name	Description	Type
<u>lead_key</u>	Primary Key (PK): Unique identifier for each lead record.	PK
<u>mql_id</u>	Degenerate Dimension: Marketing Qualified Lead identifier.	Degenerate

Column Name	Description	Type
<u>seller_key</u>	Foreign Key (FK): Links to the <u>dim_seller</u> table.	FK
<u>lead_source_key</u>	Foreign Key (FK): Links to the <u>dim_lead_source</u> table.	FK
<u>business_segment_key</u>	Foreign Key (FK): Links to the <u>dim_business_segment</u> table.	FK
<u>first_contact_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, representing the date of the first contact with the lead.	FK
<u>won_date_key</u>	Foreign Key (FK): Links to the <u>dim_date</u> table, representing the date the lead was converted (nullable).	FK
<u>is_won</u>	A flag (1 for won, 0 for not won) indicating if the lead was successfully converted into a sale.	Boolean
<u>has_company</u>	A flag (1 if the lead has an associated company, 0 otherwise).	Boolean
<u>has_gtin</u>	A flag (1 if the lead has a Global Trade Item Number, 0 otherwise).	Boolean
<u>average_stock</u>	The average stock declared by the seller associated with the lead.	Numeric
<u>declared_product_catalog_size</u>	The declared size of the product catalog by the seller.	Numeric
<u>declared_monthly_revenue</u>	The declared monthly revenue by the seller.	Numeric

4.2. Dimension Tables

4.2.1. dim_date

A conformed dimension that provides comprehensive temporal attributes, enabling flexible analysis across various time granularities.

Column Name	Description	Type
<u>date_key</u>	Primary Key (PK): An integer representation of the date (YYYYMMDD), serving as the unique identifier.	PK
<u>full_date</u>	The complete date in YYYY-MM-DD format.	Date

Column Name	Description	Type
<u>day</u>	The day of the month (1-31).	Numeric
<u>month</u>	The month of the year (1-12).	Numeric
<u>year</u>	The year.	Numeric
<u>quarter</u>	The quarter of the year (1-4).	Numeric
<u>weekday</u>	The full name of the day of the week (e.g., 'Monday').	Text

4.2.2. dim_customer

Stores detailed information about customers, allowing for segmentation and analysis based on customer demographics and identifiers.

Column Name	Description	Type
<u>customer_key</u>	Primary Key (PK): Unique identifier for each customer in the data warehouse.	PK
<u>customer_id_nk</u>	Natural Key (NK): The original customer identifier from the OLTP system.	Text
<u>customer_unique_id</u>	A unique identifier for customers across multiple orders, useful for tracking individual customers.	Text
<u>city</u>	The city where the customer resides.	Text
<u>state</u>	The state where the customer resides (two-letter abbreviation).	Text
<u>zip_code_prefix</u>	The first five digits of the customer's zip code.	Numeric

4.2.3. dim_seller

Contains information about sellers, enabling analysis of seller performance and geographical distribution.

Column Name	Description	Type
<u>seller_key</u>	Primary Key (PK): Unique identifier for each seller in the data warehouse.	PK
<u>seller_id_nk</u>	Natural Key (NK): The original seller identifier from the OLTP system.	Text
<u>city</u>	The city where the seller is located.	Text
<u>state</u>	The state where the seller is located (two-letter abbreviation).	Text
<u>zip_code_prefix</u>	The first five digits of the seller's zip code.	Numeric

4.2.4. dim_product

Provides detailed information about products, supporting analysis by product category, physical attributes, and other product-related characteristics.

Column Name	Description	Type
<u>product_key</u>	Primary Key (PK): Unique identifier for each product in the data warehouse.	PK
<u>product_id_nk</u>	Natural Key (NK): The original product identifier from the OLTP system.	Text
<u>category_name</u>	The product category name in Portuguese.	Text
<u>category_name_english</u>	The product category name translated into English.	Text
<u>weight</u>	The product's weight in grams.	Numeric
<u>length</u>	The product's length in centimeters.	Numeric
<u>product_height</u>	The product's height in centimeters.	Numeric
<u>product_width</u>	The product's width in centimeters.	Numeric
<u>photos_qty</u>	The quantity of photos available for the product.	Numeric

4.2.5. dim_payment_type

Lists various payment types used in transactions, enabling analysis of payment method preferences.

Column Name	Description	Type
<u>payment_type_key</u>	Primary Key (PK): Unique identifier for each payment type.	PK
<u>payment_type</u>	The name of the payment method (e.g., 'credit_card', 'boleto').	Text

4.2.6. dim_lead_source

Captures information about the origin of marketing leads, crucial for evaluating marketing campaign effectiveness.

Column Name	Description	Type
<u>lead_source_key</u>	Primary Key (PK): Unique identifier for each lead source.	PK
<u>landing_page_id</u>	The identifier of the landing page where the lead originated.	Text
<u>origin</u>	The source or channel from which the lead was acquired (e.g., 'social', 'paid_search').	Text

4.2.7. dim_business_segment

Categorizes leads by business segment and type, supporting targeted marketing and sales strategies.

Column Name	Description	Type
<u>business_segment_key</u>	Primary Key (PK): Unique identifier for each business segment.	PK
<u>business_segment</u>	The business segment associated with the lead.	Text
<u>lead_type</u>	The type of lead (e.g., 'industry', 'enterprise').	Text

5. ETL Pipeline Design

The ETL (Extract, Transform, Load) pipeline is implemented in Python, leveraging pandas for data manipulation and SQLAlchemy for database interactions. The pipeline is designed to be modular, robust, and reproducible.

5.1. Extraction

Data is extracted from the source SQLite database (olist.sqlite). Each OLTP table is read into a pandas DataFrame. This step ensures that all necessary raw data is available for subsequent transformations.

5.2. Transformation

The transformation phase involves several key steps to conform the raw data into the Star Schema dimensions and facts:

- **Date Dimension (dim_date):** A comprehensive date dimension is generated programmatically, covering the entire date range present in the dataset. This dimension includes attributes like day, month, year, quarter, and weekday to facilitate time-based analysis.
- **Dimension Tables:**
 - **Surrogate Keys:** New surrogate keys (_key) are generated for each dimension table (e.g., customer_key, product_key).
 - **Natural Keys:** Original identifiers from the OLTP system are retained as natural keys (_nk) for traceability.
 - **Attribute Selection and Renaming:** Relevant attributes are selected from the source tables, and columns are renamed to align with the data warehouse naming conventions.
 - **Data Cleaning:** Basic cleaning, such as handling duplicates for dim_payment_type, dim_lead_source, and dim_business_segment, is performed.
 - **Product Category Translation:** The product_category_name_translation table is used to enrich dim_product with English category names.
- **Fact Tables:**
 - **Joining Dimensions:** Fact tables are constructed by joining relevant OLTP tables and then linking to the newly created dimension tables using their surrogate keys.
 - **Degenerate Dimensions:** Original transaction identifiers (order_id, mql_id, order_item_id) are included directly in the fact tables as degenerate dimensions.
 - **Calculated Measures:** New measures are calculated as per the requirements:
 - fact_sales: quantity (set to 1 per item), unit_price, total_price.
 - fact_delivery: delivery_time_days, delivery_delay_days, is_delayed.
 - fact_reviews: has_comment.

- fact leads: is won.
- **Date Key Mapping:** Date columns from the OLTP system are converted into integer date_key values to link with dim_date.
- **Null Handling:** Null values in date columns are handled by converting them to a default 0 for date_key to maintain referential integrity with dim_date (assuming 0 is a placeholder for unknown dates in dim_date).

5.3. Loading

The transformed data is loaded into the target PostgreSQL database. For demonstration purposes, a local SQLite database (olist_dwh.sqlite) is used to simulate the PostgreSQL environment. The if_exists='replace' option is used during loading for development, but in a production environment, append or upsert strategies would be employed for incremental loads.

6. PostgreSQL DDL and Indexing

The Data Definition Language (DDL) script (olist_dwh.sql) is provided to create all tables in the PostgreSQL database. It includes:

- **Primary Keys (PKs):** Defined for all dimension and fact tables.
- **Foreign Keys (FKs):** Established between fact tables and dimension tables to enforce referential integrity and define relationships.
- **Data Types:** Appropriate PostgreSQL data types are used for each column.
- **Indexes:** Non-clustered indexes are created on foreign key columns in fact tables and natural key columns in dimension tables to optimize query performance, especially for joins and lookups.

7. Sample Analytical Queries

To demonstrate the analytical capabilities of the Data Warehouse, a set of sample SQL queries (analytical_queries.sql) is provided. These queries address common business questions:

- **Sales trends over time:** Analyzing monthly total revenue and order counts.
- **Top customers:** Identifying customers with the highest total spending.
- **Revenue by product/category:** Breaking down revenue by product categories.
- **Delivery performance:** Calculating average delivery delays and delay rates over time.
- **Lead conversion rate:** Measuring the effectiveness of marketing efforts by lead source.

8. Key Decisions, Assumptions, and Trade-offs

8.1. Key Decisions

- **Star Schema Adoption:** Chosen for its simplicity, ease of understanding, and optimized query performance for analytical reporting.
- **Degenerate Dimensions:** order_id, order_item_id, and mql_id are included directly in fact tables to preserve transactional detail without creating unnecessary dimensions.
- **Conformed Date Dimension:** A single dim_date table serves all fact tables, ensuring consistent date-based analysis.
- **Python for ETL:** Python with pandas and SQLAlchemy was selected for its flexibility, extensive data manipulation capabilities, and readability.

8.2. Assumptions

- **Data Quality:** Assumed that the source SQLite data is largely clean and consistent, requiring minimal data cleansing beyond what's handled in the ETL script.
- **Date Range:** The dim_date table is generated for a fixed range (2016-01-01 to 2019-12-31), covering the known data in the Olist dataset. This can be extended if new data outside this range is introduced.
- **PostgreSQL Environment:** Assumed a PostgreSQL database will be available for deployment, with appropriate permissions for table creation and data loading.
- **Seller Mapping in fact_delivery:** For orders with multiple items from different sellers, the fact_delivery table currently picks the seller_id of the first item. A more complex logic might be needed if delivery performance needs to be attributed to multiple sellers per order.

8.3. Trade-offs

- **Redundancy in Degenerate Dimensions:** While degenerate dimensions simplify queries by avoiding joins to small dimensions, they introduce a slight redundancy of data within the fact tables.
- **ETL vs. ELT:** The current approach is more ETL-centric (extract, transform, load). For very large datasets, an ELT approach (extract, load, transform) might be considered, where raw data is loaded into a staging area in the DWH first, and transformations are performed using SQL within the DWH for better scalability.
- **Simulated PostgreSQL:** Due to environment constraints, PostgreSQL loading is simulated using SQLite. This means actual PostgreSQL-specific optimizations (e.g., partitioning) are not demonstrated in the loading script but are included in the DDL.